

Wie niet sterk is, moet slim zijn.

“Those who are not strong must be smart.” — Dutch proverb

Pauli propagation at 10^7 terms

How a GF(2)-linear hash turned a memory-bound simulator strong *and* smart

ACT 1

Sterk

Brute force against the exponential wall

Why Pauli propagation

The question. $\langle O \rangle$ after a deep circuit on 100+ qubits.

- State vectors: 2^n amplitudes — exact to 30 qubits, then nothing.
- Tensor networks: pay for entanglement, not for qubits.
- **Pauli propagation:** pay for the *operator*, not the state.

$$U^\dagger O U = \sum_P c_P P, \quad P \in \{I, X, Y, Z\}^{\otimes n}$$

Evolve O backwards through the gates, keep the sum sparse by **truncation**, contract with ρ_0 at the end.

Exactness traded for *sparsity*: the sum stays small when most of the 4^n coefficients are negligible — deep, noisy, or weakly non-Clifford circuits.

Reproduces the 127-qubit IBM “utility” observables (Kim et al. 2023); bit-exact against `stim` at the Clifford points.

Pauli strings as bits: the symplectic encoding

Each qubit's Pauli is a bit pair (x, z) :

I	X	Z	Y
(0, 0)	(1, 0)	(0, 1)	(1, 1)

qubit	0	1	2	3	
Pauli	X	I	Z	Y	
x	1	0	0	1	0b1001
z	0	0	1	1	0b1100

A string on n qubits is two words: x : $[u64; W]$, z : $[u64; W]$, $W = \lceil n/64 \rceil$.

- 128 qubits $\rightarrow$ **32 bytes** per string, no pointers, Copy
- product: $(x_1 \oplus x_2, z_1 \oplus z_2)$ and a phase i^k
- commute iff $x_1 \cdot z_2 + z_1 \cdot x_2 = 0 \bmod 2$ — two ANDs and a popcount

A sum of Pauli strings is an array of (x, z, c) rows: 48 bytes per term at 128 qubits. Remember that number.

Clifford gates permute Pauli strings

A Clifford gate maps one Pauli string to **one** Pauli string (up to sign):

gate	in	out
H	X, Z	Z, X
S	X, Y	$Y, -X$
CNOT	X_c	$X_c X_t$
CNOT	Z_t	$Z_c Z_t$

In bits: the key changes only on the gate's **support**, by an XOR with a vector from a small **delta set** D .

- Term count is **invariant**: m in, m out.
- One table lookup per term: a permutation of the sum.
- Truncation never triggers.

Clifford circuits are free. A stabilizer simulator does this for a single string in $O(n^2)$; we do it for 10^7 strings at once.

Beyond Clifford: one string becomes two

A Pauli rotation $R = \exp(-i\theta P/2)$ acting on a term Q :

$$R^\dagger Q R = \begin{cases} Q & \text{if } [P, Q] = 0 \\ \cos \theta Q + \sin \theta (iPQ) & \text{if } \{P, Q\} = 0 \end{cases}$$

- the anticommuting part of the sum **doubles** every layer: up to 2^d terms after d kicks
- iPQ is again a Pauli string — an XOR of keys

Truncation keeps it tractable: drop $|c| < \varepsilon$, drop weight $> k$, keep the top N . Noise shrinks high-weight terms and makes the problem **easier**.

$$c \cdot Z_1 Z_2$$

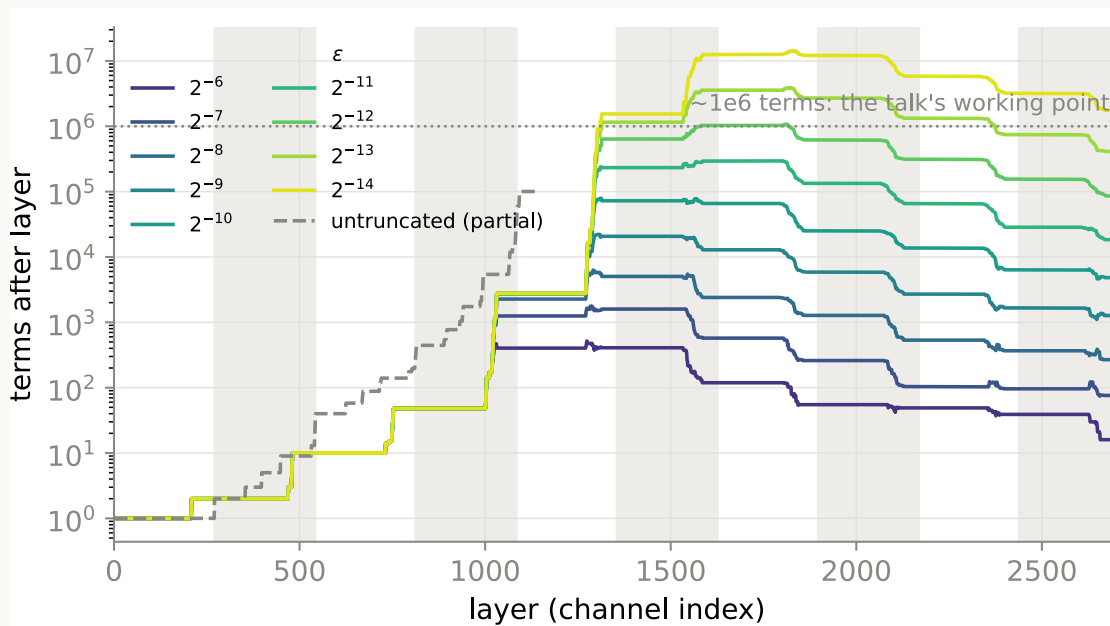
$R_X(\theta)$ on qubit 1: X_1 anticommutes with Z_1

$$c \cos \theta \cdot Z_1 Z_2$$

$$c \sin \theta \cdot Y_1 Z_2$$

next R_X layer: both split again $\rightarrow 4, 8, 16, \dots$

The exponential wall, on the talk's circuit



127-qubit heavy-hex kicked Ising, $\theta_{ZZ} = -\pi/2, \theta_h = 5\pi/16$, observable Z_{62} , Heisenberg, 10 Trotter steps. Terms after each channel; grey bands alternate steps.

- 127 qubits, 144 couplings, 271 rotations per step; the light cone needs five steps to fill the lattice
- working point for everything that follows: **10 steps, 2710 rotations, $\epsilon = 2^{-12}$** : peak $1.07 \cdot 10^6$ terms, half the layers above 10^5
- **$\approx 3 \cdot 10^9$ term updates per run**

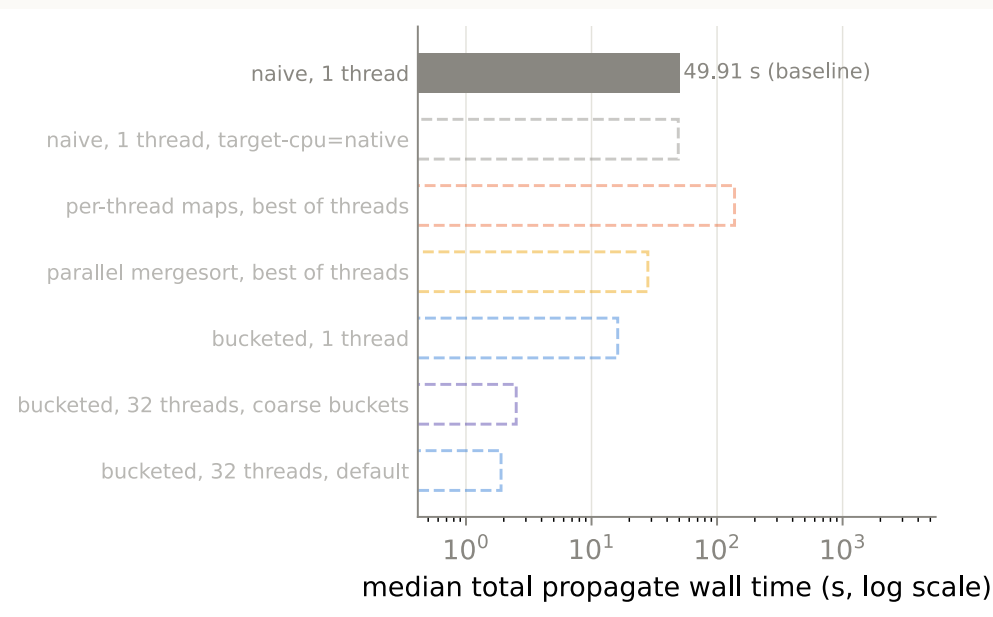
Strong means 10^9 – 10^{10} updates per second. Fine — a CPU does 10^{11} simple ops per second.

The naive algorithm: a hash map per layer

```
for gate in circuit.reversed():
    out = {}
    for (P, c) in sum:
        for (P', c') in gate.apply(P,
c):
            out[P'] += c'
    sum = {P: c for P, c in out
            if |c| >= eps}
```

Or with arrays: emit all outputs, **sort**, **merge** equal keys, filter.

This is the engine's shipped small-sum path: hashbrown + FxHash, Channel::apply per term.



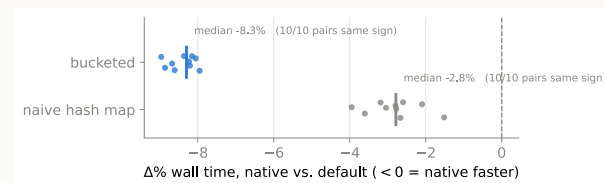
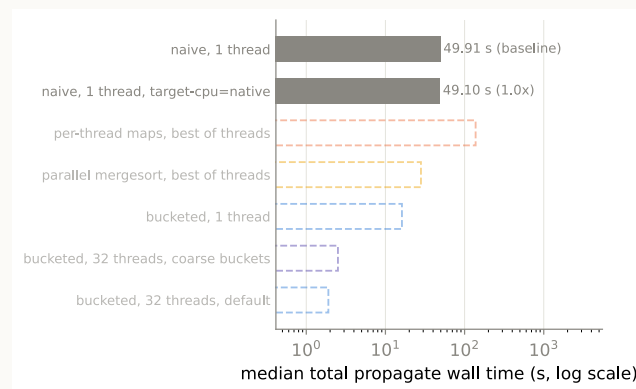
Time to propagate the 2710-channel circuit (peak $1.07 \cdot 10^6$ terms). This chart returns after every idea.

Being strong: make the kernel fast

What one would do first:

- a fast non-cryptographic hash (FxHash, not SipHash)
- popcount and XOR: already single instructions
- let LLVM vectorize: `-C target-cpu=native`
- fat LTO, one codegen unit — already on

Paired, alternated A/B: a real, consistent gain — 3 % on the hash map, 8 % on the sorted engine. And a hundred times too small.



Paired runs, alternated abba, one thread.

ACT 2

The wall is memory

Why faster arithmetic does not help

Count the bytes, not the flops

One term: $2 \cdot 16 \text{ B key} + 16 \text{ B coefficient} = \mathbf{48 \text{ B}}$.

One layer at $m = 10^6$ terms, fanout 2:

- read the sum: $m \cdot 48 \text{ B}$
- write the outputs: $2m \cdot 48 \text{ B}$
- sort or probe them: $2m \cdot 48 \text{ B}$, read and written again
- write the merged result: $m \cdot 48 \text{ B}$

≈ **300–400 MB per layer**, 0.5 TB per run. The arithmetic per term: a few XORs, one multiply — < **1 ns**.

Two ways to lose:

- **Hash map** = random access. Each probe into a 50 MB table is a cache miss: 100 ns, one at a time.
- **Sort** = streaming. Each pass runs at memory bandwidth: 10 GB/s per core → 30–40 ms per layer just to move the bytes.

Either way the core waits on memory.

What the machine actually delivers

Measured on the reference host (2× Xeon Gold 6244, 16 cores, STREAM-style, `crates/membench`):

placement	read	write	copy
1 core	11.3	10.1	9.5
1 core, other socket's memory	7.8	7.2	5.5
8 cores, one socket	39.0	18.6	35.6
16 cores, both sockets	45.0	25.3	40.0
32 threads (SMT)	48.8	23.1	33.8

GB/s. Spec sheet says 141 GB/s per socket — only 2 of 6 memory channels are populated.

- hyperthreads add **no** bandwidth
- the second socket adds **15–25 %**, not 2×
— remote reads run at 7.8 GB/s
- write bandwidth is **half** of read

Strong is 45 GB/s, shared by everyone. At 48 B/term that is a hard ceiling of 10^9 term-moves per second — before any algorithm.

Three levers, and a hard limit

Fewer bytes

Compact keys, no pointers,
structure-of-arrays, coefficients
that are not boxed.

More work per byte

Do everything you need with a
term while it is in cache.
Locality, not clock speed.

More bandwidth

More cores, more sockets,
more nodes — **and** their caches
and their memory capacity.

The other wall: 10^7 terms is 0.5 GB; the sort scratch of a global algorithm is several times that. At 10^8 terms a single node runs out of memory before it runs out of time.

So: multithreading — not for the cores, for the **bandwidth** and the **memory** they bring.

What I tried first (a reconstruction)

A. Per-thread dictionaries

- split the terms in chunks, one hash map per thread
- apply the gate locally
- **merge all maps into one** at the end of the layer
- truncate, repeat

B. Parallel mergesort

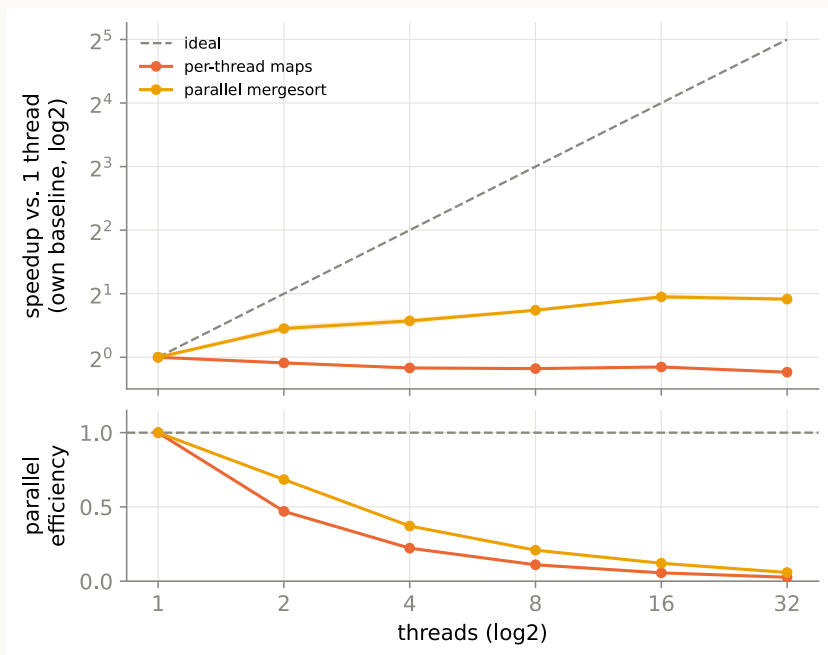
- one flat array of (x, z, c) rows
- every thread appends its outputs
- **parallel sort** of the whole array, then a merge pass for equal keys

Both are what a competent engineer writes in an afternoon. Both share a problem:

The **merge** is global. It touches every output row once more, through memory, and is serial or nearly so. Amdahl does the rest.

The Julia-era attempts are not in this repository; these two are reimplemented in Rust on the same circuit and gated against the reference engine to 10^{-9} . The numbers on the next slide are fresh, the memory of “never past 2× on 32 cores” is old.

Strong, not smart: 32 cores, a factor two



Reconstructed baselines on the talk's circuit, 1 → 32 threads (16 cores). Bands: min-max of repeated runs.

- mergesort: **1.9×** at 16 threads, then flat
- per-thread maps: **never faster than one thread** — the final merge re-inserts every term, which is the naive algorithm, once more, serially
- memory per term goes **up** (copies): 280 and 830 B against 240 B for the plain map

Amdahl: a serial fraction s caps the speedup at $1/s$. A merge that is half the layer caps you at 2×, whatever the core count.

We were strong. We were not smart.

Every attempt kept **one global order** — one map, one sorted array — and paid for it with one global merge. The parallel part was easy; the sequential part was the algorithm.

The project rested for a while.

ACT 3

Sterk én slim

GF(2)-linear hash bucketing

The idea: a hash that respects XOR

Pick a random binary matrix $H \in \text{GF}(2)^{b \times 2n}$ and partition the sum into $B = 2^b$ buckets by

$$h(v) = Hv \pmod{2}.$$

A gate changes a key by an XOR: $v \rightarrow v \oplus d, d \in D$, $|D| \leq 16$ for any 2-qubit gate.

Linearity:

$$h(v \oplus d) = h(v) \oplus h(d).$$

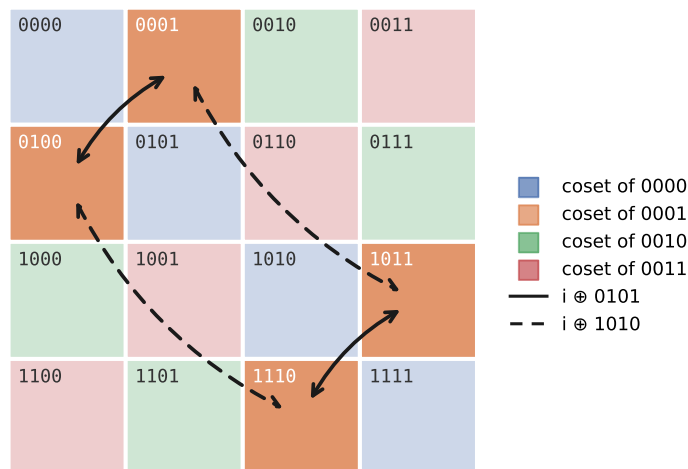
So every term in bucket β lands in bucket $\beta \oplus h(d)$
— **known before any term is touched.**

- a bucket's outputs go to $\leq 2^{\dim D}$ statically known buckets
- a bucket's inputs come from the same small set
- two equal output keys have equal hashes:
duplicates never straddle buckets

Compare: a general hash scatters a gate's outputs over all B buckets, and the coordinate projection “bucket by qubit 0's bits” collapses under weight truncation — everything is in bucket 0.

Consequences: closed tasks, local merges

16 buckets, $h(D)$ spanning $\{0101, 1010\}$: four closed cosets
arrows = one coset's complete data movement



16 buckets, $h(D)$ spanning two vectors: four closed cosets. Arrows: one coset's entire data movement.

The image of D spans a subspace $S = \text{span}(h(D))$; its **cosets** partition the buckets.

- a coset's outputs read only the **same** coset → a **closed task**
- write-disjoint by construction: **no locks, no atomics, no barriers**
- dedup is **bucket-local**: sort and merge a cache-sized run
- **no global sort** anywhere in the loop

Rayon work-steals cosets. That is the whole parallel runtime.

Load balance comes from randomness

A dense random H is a **universal hash family**:

$$\max_{\beta} |\text{bucket}_{\beta}| \leq \frac{m}{B} + O\left(\sqrt{\frac{m \log B}{B}}\right)$$

with high probability, **independently of the input's structure**.

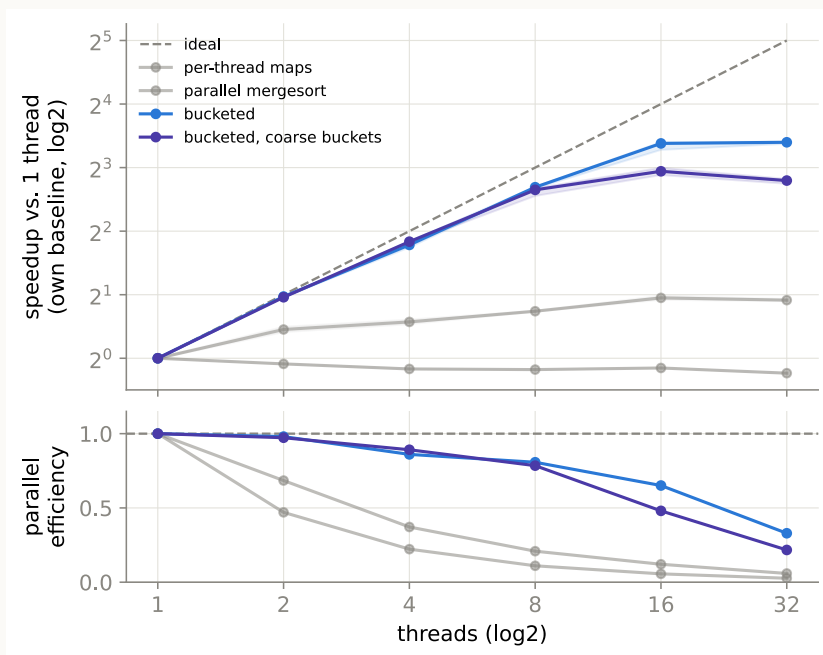
Weight-truncated sums are extremely structured (mostly low weight, identities everywhere) — a coordinate-based partition would put everything in a handful of buckets. The random H does not care.

$\text{rank}(H|_D) = \dim D$ with prob. $\geq 1 - 2^{\dim D - b}$

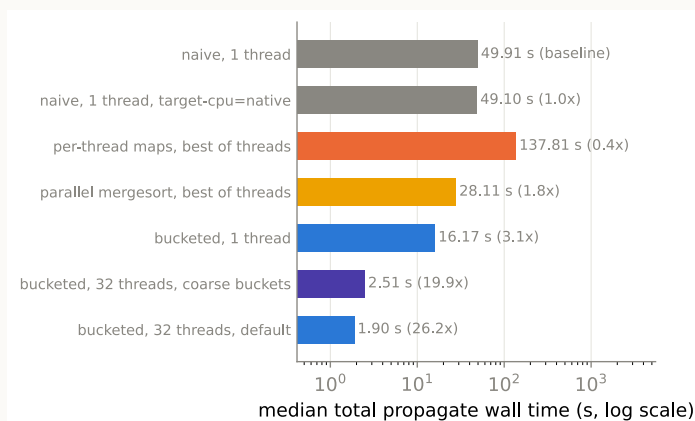
- full rank: outputs spread over $2^{\dim D}$ buckets, gather runs arrive presorted
- deficient rank (10 % of 2-qubit placements at $B = 128$): still correct, sort works harder

Bucket count follows the sum: target **1024 terms per bucket**, floor 128 buckets, grow-only.
Refining splits every bucket in two, in order, in one parity pass — no re-sort.

Results on the kicked-Ising circuit

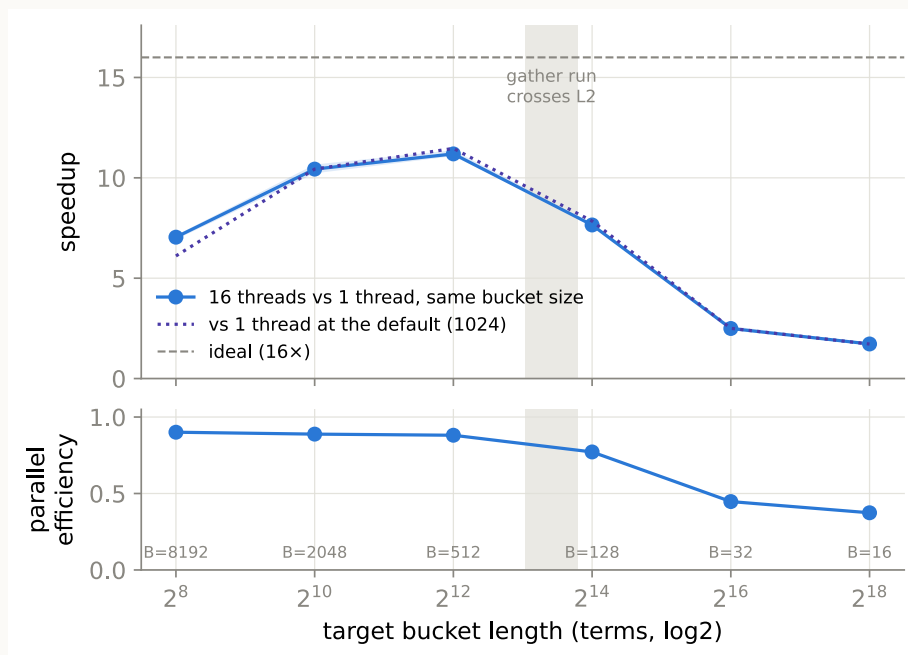


Speedup vs own single-thread run; the two reconstructions greyed. 16 physical cores, 32 hyperthreads.



- single-threaded already **3×** faster than the map: no global structure to maintain
- 16 threads: **10.5×** on this circuit (11–13× on pure rotation layers), against $\leq 2\times$ before; hyperthreads add nothing

The bucket size decides whether cores help at all



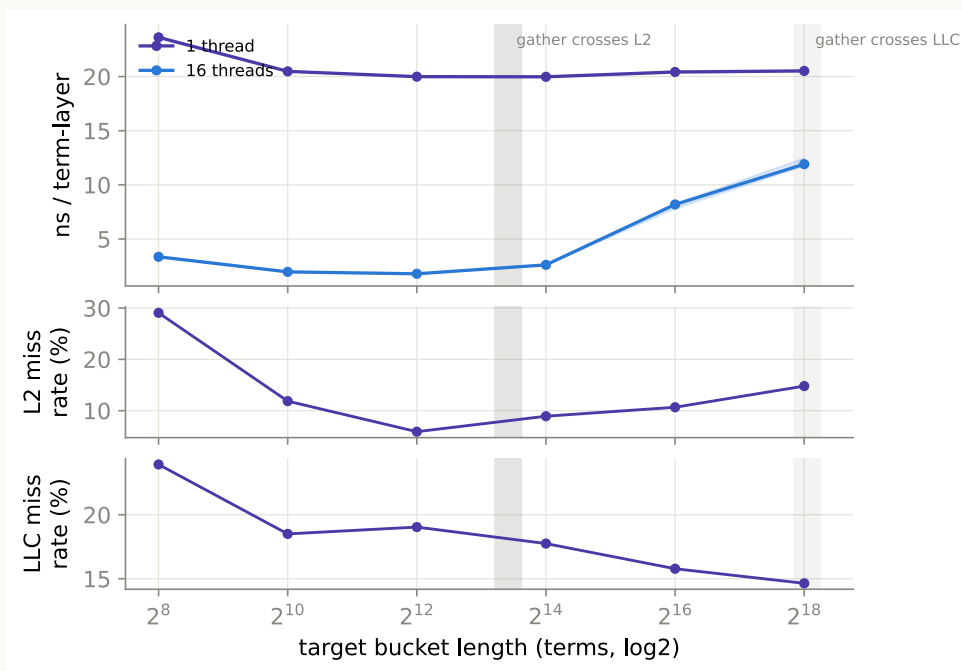
Same code, same circuit, 16 threads. Top: speedup over one thread. Bottom: worker busy time / (loop wall $\times$ threads), from the engine's phase counters.

Same 16 cores: **11x or 2x**, depending only on how many terms share a bucket.

Past 10^4 terms per bucket two things happen: the gather run stops fitting L2 (1 MiB per core), and there are fewer cosets than threads (32 buckets $\rightarrow$ 16 tasks). The efficiency panel shows how much is starvation; the rest is the memory system.

Cores bring **cache**, not just clocks: 16 cores are 16 MiB of private L2. Buckets are what let the working set live there.

The mechanism: bucket size against L2



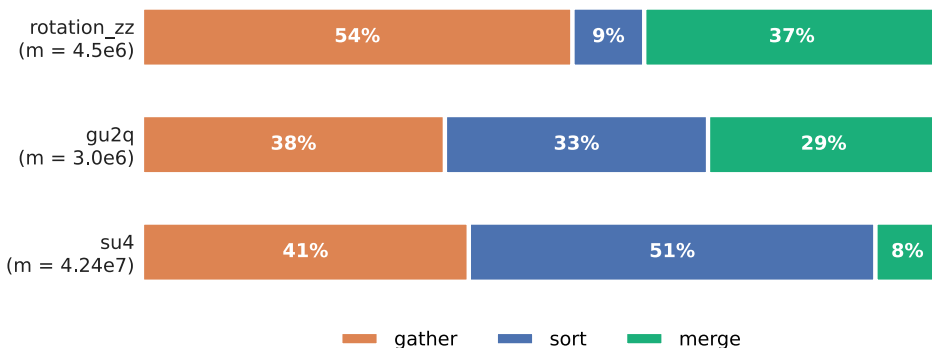
Per-term cost at 1 and 16 threads, and single-thread L2 / LLC demand-miss rates (perf stat), vs bucket size.

- **one thread does not care:** 20 ns per term-layer from 2^8 to 2^{18} terms per bucket — latency-bound either way
- **sixteen threads care a lot:** they share one memory system, and only cache-resident runs keep it out of the loop
- the **gather run**, fanout $\times$ bucket, is what must fit: $1024 \times 48 \text{ B} \times 2 \approx 100 \text{ KB}$ against 1 MiB of L2
- dense 2-qubit unitaries have fanout 16 $\rightarrow$ 750 KB runs $\rightarrow$ the **write ceiling** at 16 threads (fact sheet)

Same algorithm, same bytes — only the **order** in which they are touched changed.

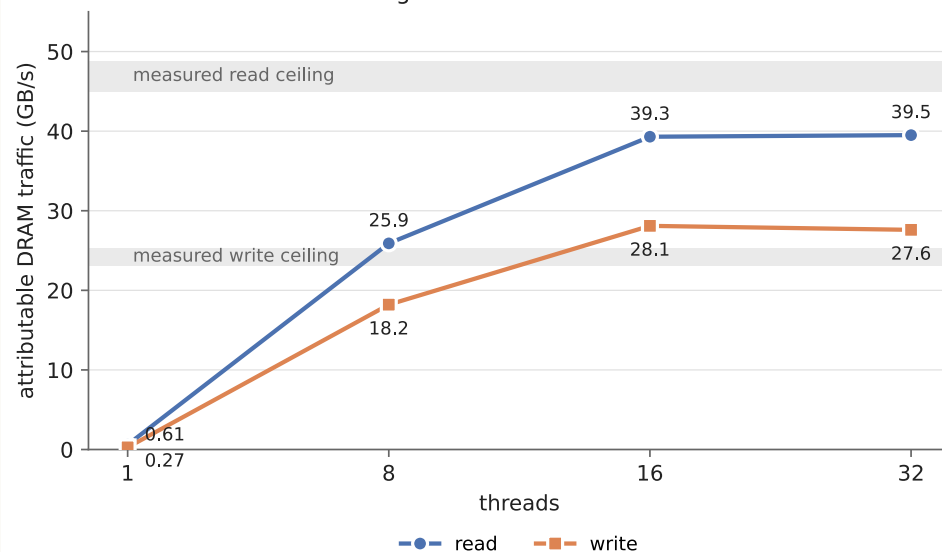
Where the time goes now

Share of worker busy time, one thread



Share of worker busy time per phase, three layer classes, 1 → 32 threads (fact sheet, 2026-09-01).

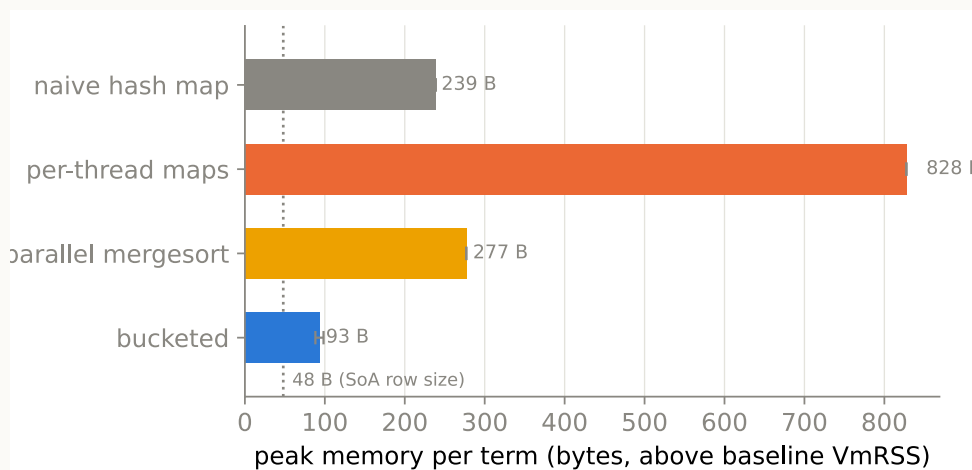
su4 sits on the write ceiling from 16 threads



Achieved DRAM traffic against the measured ceilings.

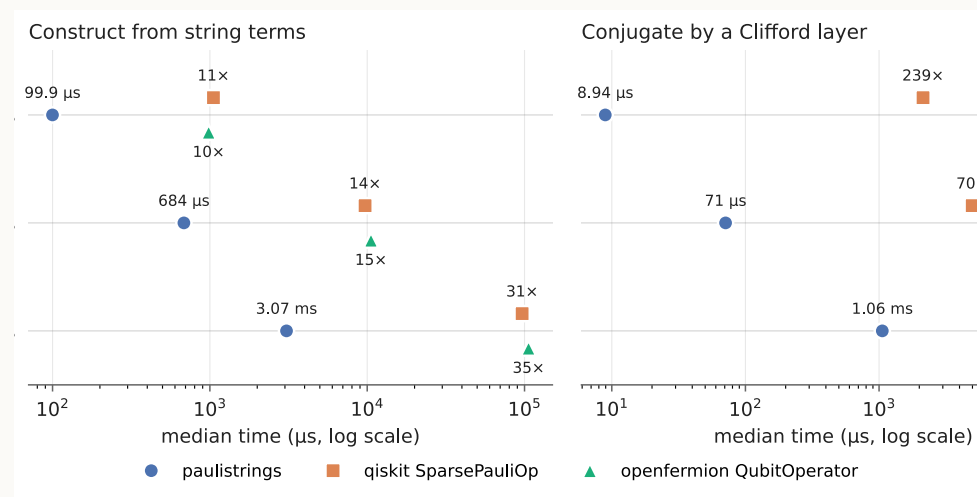
Sparse layers are **latency**-bound at half the write ceiling even at 32 threads (70–90 % of modelled traffic is served from cache); only dense 2-qubit PTMs hit the write ceiling. The single-thread byte model over-counts DRAM traffic 2.5–13×: the design's whole point.

Footprint, and where it stands



Peak RSS above the process floor, per peak term, one process per variant.

Structure-of-arrays, no pointers, capacity retained across layers:
the steady state of a propagation allocates nothing.



Construction and Clifford conjugation vs `qiskit.SparsePauliOp`
and `openfermion.QubitOperator`.

Single-threaded vs `PauliPropagation.jl`: 2-3 $\times$ faster
above a few thousand terms, 5 $\times$ less memory per term. Below
that, its hash map wins — and the engine switches to one.

CODA

The gift that keeps on giving

One structure, many wins

One structure, many wins

The partition on h was built for parallelism. It also gave:

- **locality** — cache-sized runs, superlinear scaling
- **footprint** — bucket-local scratch, no global sort buffers
- **GPU-ready** buffers — $\#[\text{repr}(C)]$ SoA columns, fixed fanout

And it is a **communication plan**. Partition on h across ranks: a layer is a sparse, statically known exchange, computed from H and D before any term is touched. No key is ever split across ranks.

rank 0
buckets 00xx

rank 1
buckets 01xx

rank 2
buckets 10xx

rank 3
buckets 11xx

a gate with $h(D) \subset \text{span}(0100, 1000)$ exchanges only between ranks in the same coset — most gates touch no rank boundary at all

NUMA is the same problem one level down: two sockets, two “ranks”, remote reads at 7.8 GB/s.

Open threads, and the proverb the right way round

- **NUMA-aware placement.** A static coset→worker map lost 1.25–1.9× to work stealing (stragglers). The smart version has to steal *within* a socket first.
- **Dense-PTM write ceiling.** Fanout-16 gather runs at 16 threads saturate the write path; smaller runs or fused passes.
- **Distributed prototype.** The exchange plan exists on paper; the buckets are already the message boundaries.
- **Rank-deficient hash seeds,** a channel-aware bucket floor, more channel types.

Sterk: 32 cores, 45 GB/s, every byte of L2.

Slim: an algebraic structure — a linear hash — that tells you where everything goes before you move it.

Wie sterk is, moet ook slim zijn.

github.com/lkdvos/paulistrings-rs — Rust core, Python bindings, all figures reproducible from the repository.

Appendix

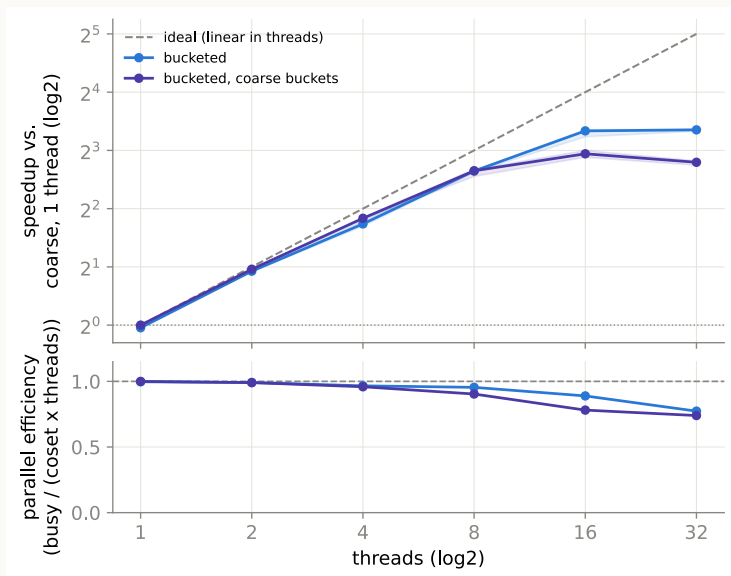
Negative results and reproduction

Things that did not work (and are written down)

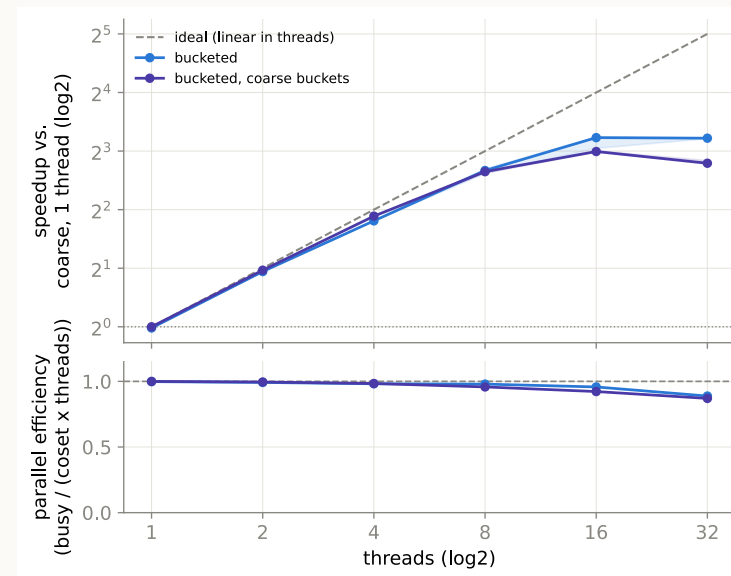
- **Bucket by support bits and concatenate** (v0.1 design): provably wrong — buckets interleave, they do not concatenate in sorted order. Four-term counterexample. → led to the linear hash.
- **Static coset → worker placement** for NUMA locality: 1.25–1.9× slower than work stealing in 7 of 8 cells. Stragglers cost more than locality recovers.
- **Recompute-in-merge borrowing, segment-copy merging, interleaved transient key layout**: three gather/merge variants, all rejected by paired A/B; one narrow survivor (–3 to –11 %).
- **Reserving a “safe upper bound” for merge output**: 2.5× worse peak RSS on high-collision channels — the merge’s job is to shrink.
- **LTO code-layout effects are real**: an `#[inline]` hint moves the merge kernel by 6–34 %. Every attribute in `engine/merge.rs` is A/B-verified in both directions.

research/notes/ in the repository: one note per negative result, with the numbers.

Appendix: coarse vs default buckets across thread counts



$1.07 \cdot 10^6$ peak terms; coarse = 16 384 terms per bucket (64 buckets).



$3.9 \cdot 10^6$ peak terms ($\epsilon = 2^{-13}$); coarse = 16 384 terms per bucket (256 buckets).

Normalised to the coarse single-thread run. With 64–256 buckets the coarse arm still has enough cosets; the collapse needs ≤ 32 . No superlinear regime on this host and circuit: 9.5–10.5× on 16 cores either way.

Reproducing every number in this talk

- **Host:** ccqlin038 — 2× Xeon Gold 6244 @ 3.6 GHz, 16 cores / 32 threads, 2 NUMA nodes, 1 MiB L2 per core, governor powersave. Bandwidth ceilings from `scripts/bandwidth.sh`.
- **Circuit:** `examples/common/circuits.py::heavy_hex_kicked_ising(127, 10, 5 $\pi$ /16, - $\pi$ /2)`; Rust port in `presentation/bench/src/workload.rs`. Observable Z_{62} , Heisenberg, `CoefficientThreshold(2-12)`.
- **Baselines:** `presentation/bench` — naive (the shipped direct hash-map path), `threadmaps`, `mergesort`, `bucketed`; each gated against propagate to 10^{-9} .
- **Data:** `presentation/data/*.jsonl` with provenance headers (`commit`, `rustc`, `host`, `date`, `load`). One script per figure in `presentation/plots/`.
- **Protocol:** `RUST_LOG` unset, dedicated Rayon pool per cell, one warm-up, medians of repeated runs; paired A/B with direction consistency for anything under 10 %.
- **Engine knob** added for this talk: `PropagateOptions { target_bucket_len, min_buckets }`.